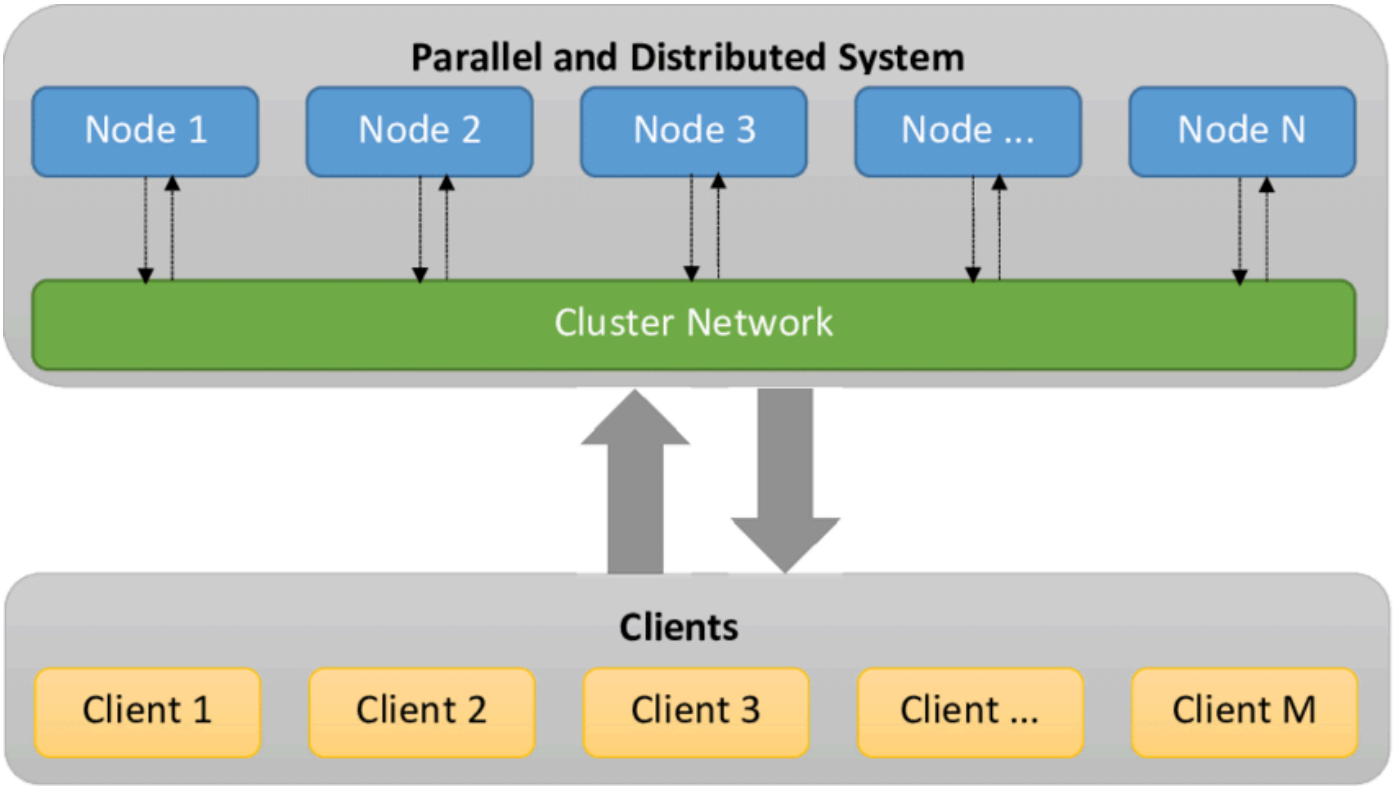
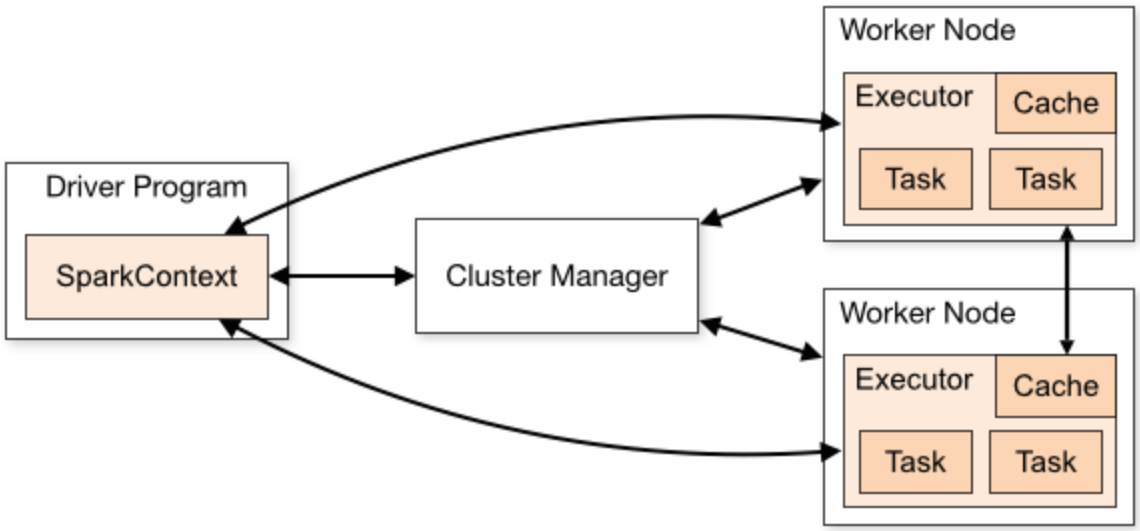


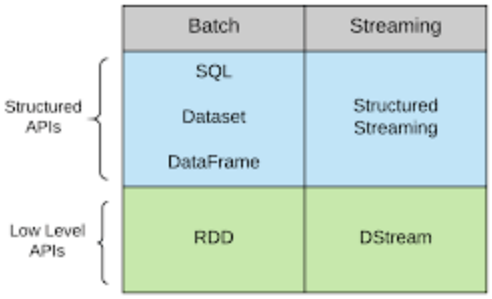
1: Spark Applications and Basic Spark Concepts

pyspark Doc

Spark provides two sets of APIs, *Structured APIs* and *low-level APIs*. The Structured APIs are designed to implement the business logic of Spark applications and they hide the Spark internals of the *low-level API*. So for us , as a ETL developer, the Structured APIs are the best starting point to dive into data processing with Spark.

Today's challenge is to write our first little Spark application to get to get a first impression of the *Structured APIs* like `DataFrame` , `Dataset` , *SQL Tables/Views* and *Structured Streaming* and to undertsand some basic concepts like lazy evaluation of transformations, and data processing actions.





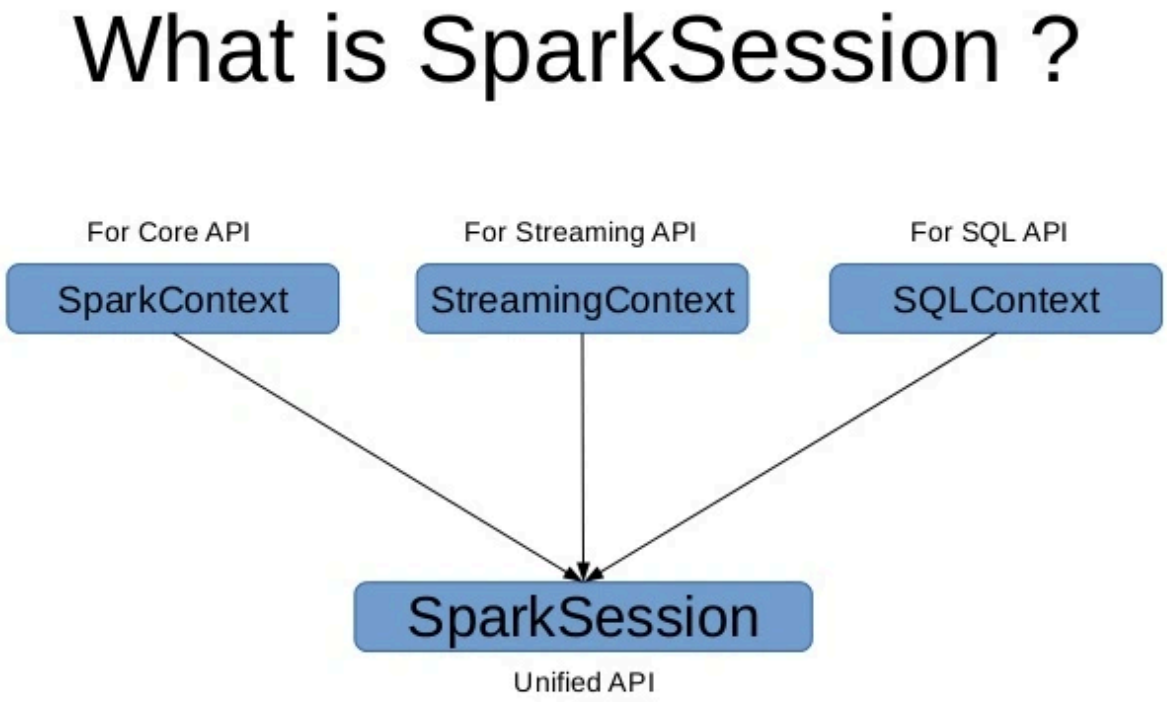
The first questions, that comes to our mind is, how to start a Spark application?

SparkSession

Starting a Spark application generates a Spark job which is controlled and mangaged by exactly one *driver process* and several *executor processes* running across the cluster nodes doing the actual computational work. The driver process is controlled by a `SparkSession` object, which is the entry point of any Spark application, so there is always a one-to-one relationship between SparkSession and Spark application.

So how can westart a Spark session?

Since we don't want to type in all the code line by line into the interactive console, our Spark application must create it's own `SparkSession` object. So every Spark application starts with something like this:



In [29]:
```
%pip install pyspark==3.5.0
```

In [30]:
```
import os
os.environ["JAVA_HOME"] = r"C:\Program Files\Eclipse Adoptium\jdk-8.0.432.6-hotspot"
```

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()
```

In [31]:
```
spark.version
```

Out[31]:
```
'3.5.4'
```

Spark UI

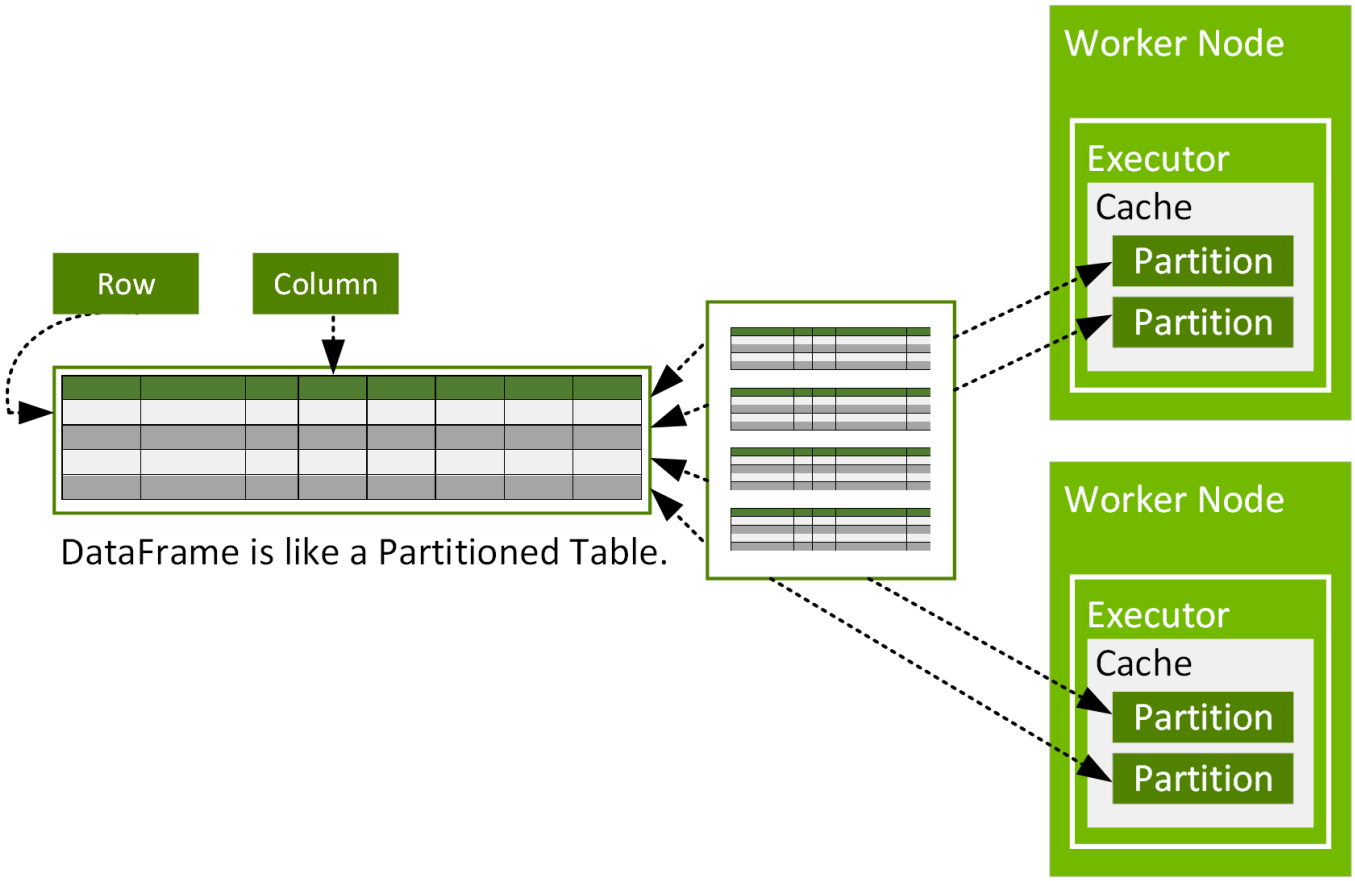
Spark provides an UI to monitor the status and progess of Spark jobs. It is available on port 4040 on the driver node in the Spark cluster. Since running Spark in local mode, i.e. all processes are running on our laptop, we can access the UI on http://localhost:4040.

After having executed the next code block in the Spark DataFrames section, the UI showed this to us.

Clicking on a jobname provides further details and us trics figures regarding the job execution icluding a graphical representation of the execution DAG (directed acyclic graph).

Spark DataFrames

run hello world example which creates our first `DataFrame` , and similar to the interactive console, the starting point is the `SparkSession` object named *spark*.



In [32]:
```
ourFirstDataFrame = spark.range(100).toDF("number")
ourFirstDataFrame.show(10)
```

```
+------+
|number|
+------+
|     0|
|     1|
|     2|
|     3|
|     4|
|     5|
|     6|
|     7|
|     8|
|     9|
+------+
only showing top 10 rows
```

Spark `DataFrame` objects look quite similar to Pandas dataframes. In fact, we can easily transform a Spark `DataFrame` into a Pandas dataframe.

But caution: This us thod should only be used if the resulting Pandas's DataFrame is expected to be small, as all the data is loaded into the driver's us mory.

In [33]:
```
pandasDF = ourFirstDataFrame.toPandas()
pandasDF.head(10)
```

Out[33]:

	number
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9

By the way, tranforming data to JSON is also easy. Each row is turned into a JSON document as one element in the returned list.

In [34]:
```
ourFirstDataFrame.toJSON().collect()
```

Out[34]:
```
['{"number":0}',
 '{"number":1}',
 '{"number":2}',
 '{"number":3}',
```

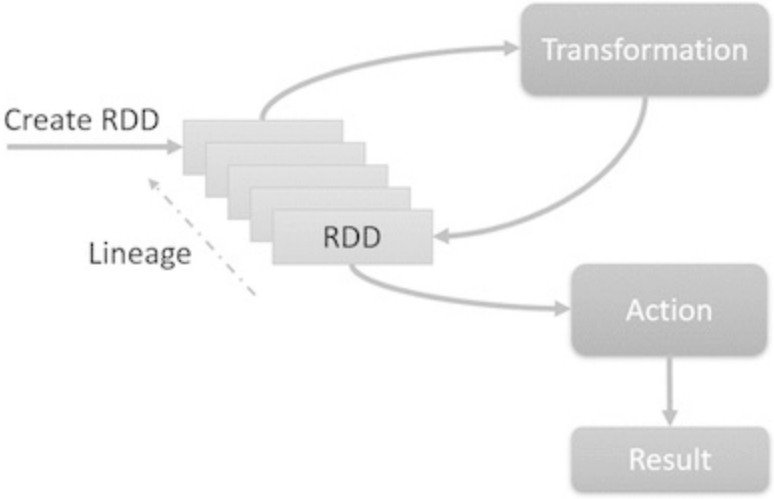
```
'{"number":4}',
'{"number":5}',
'{"number":6}',
'{"number":7}',
'{"number":8}',
'{"number":9}',
'{"number":10}',
'{"number":11}',
'{"number":12}',
'{"number":13}',
'{"number":14}',
'{"number":15}',
'{"number":16}',
'{"number":17}',
'{"number":18}',
'{"number":19}',
'{"number":20}',
'{"number":21}',
'{"number":22}',
'{"number":23}',
'{"number":24}',
'{"number":25}',
'{"number":26}',
'{"number":27}',
'{"number":28}',
'{"number":29}',
'{"number":30}',
'{"number":31}',
'{"number":32}',
'{"number":33}',
'{"number":34}',
'{"number":35}',
'{"number":36}',
'{"number":37}',
'{"number":38}',
'{"number":39}',
'{"number":40}',
'{"number":41}',
'{"number":42}',
'{"number":43}',
'{"number":44}',
'{"number":45}',
'{"number":46}',
'{"number":47}',
'{"number":48}',
'{"number":49}',
'{"number":50}',
'{"number":51}',
'{"number":52}',
'{"number":53}',
'{"number":54}',
'{"number":55}',
'{"number":56}',
'{"number":57}',
'{"number":58}',
'{"number":59}',
'{"number":60}',
'{"number":61}',
'{"number":62}',
'{"number":63}',
'{"number":64}',
'{"number":65}',
```

```
  '{"number":66}',
  '{"number":67}',
  '{"number":68}',
  '{"number":69}',
  '{"number":70}',
  '{"number":71}',
  '{"number":72}',
  '{"number":73}',
  '{"number":74}',
  '{"number":75}',
  '{"number":76}',
  '{"number":77}',
  '{"number":78}',
  '{"number":79}',
  '{"number":80}',
  '{"number":81}',
  '{"number":82}',
  '{"number":83}',
  '{"number":84}',
  '{"number":85}',
  '{"number":86}',
  '{"number":87}',
  '{"number":88}',
  '{"number":89}',
  '{"number":90}',
  '{"number":91}',
  '{"number":92}',
  '{"number":93}',
  '{"number":94}',
  '{"number":95}',
  '{"number":96}',
  '{"number":97}',
  '{"number":98}',
  '{"number":99}']
```

Ok, back to topic. The main difference bewteen Spark and Pandas is, that Pandas dataframes reside on a single machine whereas a Spark `DataFrame` ìs an abstraction of the in-memory optimized low-level API *Resilient Distributed Dataset* (`RDD`), which is designed to be split up data into partitions which can be spread across a cluster of potentially thousends of nodes. Spark `DataFrame` objects have a surprising characteristic, they are immutable once they are created. So how can data processing works when data structures are written in stone?

Lazy Evaluation, Transformations and Actions

The few lines of our code already demonstrate that the Structured API has a functional design. Since `DataFrame` objects are immutable, we have to use functions which read `DataFrame` objects as input, do some kind of data transformation and create a new `DataFrame` which again can be the input of another function to do further transformations and generating another `DataFrame` . So finally we can simply concatenate functions to create a sequence of transformations to get our desired data result.



Ok, let's do it. we want to see, if how it works. First, we want to read some data from CSV file into a dataframe. The file has a header line and we want Spark to derive the schema, i.e. name and type of the columns, from the file. Nevertheless we could also specify a schema explicitly instead of deriving it from file. Determining the schema processing time, instead of load time, is an example of the common *schema-on-read* design of Big Data architectures.

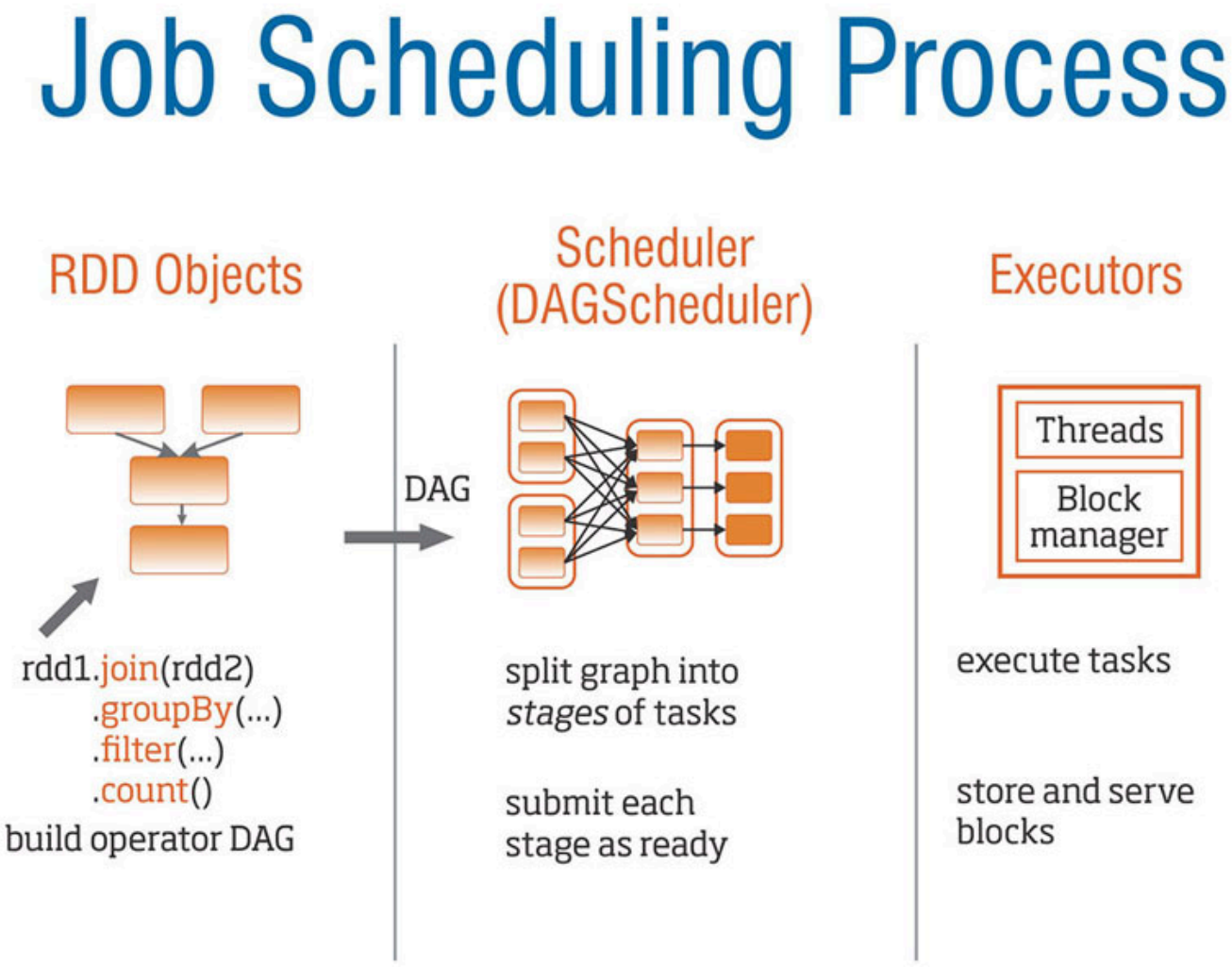
Important to keep in mind is, that the column types are not Python types (or Scala or Java types if we use another API languages). All language API commands are mapped to the Spark internal language *Catalyst* having its own types. That's why all API languages provide the same performance.

In [35]:
```
dfFlight = spark.read\
   .option("inferSchema", "true")\
   .option("header", "true")\
   .csv("./data/flight-data/2015-summary.csv")
```

Running this code doesn't show any observable result to us . That looks strange on first view. Actually, Spark hasn't done anything yet, except for deriving the schema by reading a small sample of rows. This is because Spark applies *lazy evaluation* of *transformations*, i.e. no data is moved or processed until Spark is forced to by an *action*, e.g. by calling the `write()` function.

By defining transformations, we just give Spark a set of rules describing how a given `Dataframe` should be logically transformed into a new `Dataframe` object. By calling an action, we give Spark the command to apply these transformations, process the data and provideusthe results.

The reason for the lazy evaluation approach is, that Spark first wants to know the whole story about *what* should be done effectively before it tries to determine an efficient way *how* to do this. Therefore Spark first compiles all transformations into a **logical** directed acyclic graph (DAG), than analyses this DAG, applies optimizations (e.g. predicate push-down to datasources) whenever possible and splits up the optimized **physical** DAG into stages and parallelised tasks of `RDD` manipulations before starting to execute them.



Important to note is, tha `DataFrame` objects are kept in us mory when ever possible. In contrast to MapReduce, Spark tries to avoid writing intermediate results (i.e. `DataFrame` objects) to disk by *piplining* consecutive in-memory transformations, to gain better performance.

This piplining is only possible for *narrow* transformations. These are transformations where each input partition contributes only to one output partition or where the transormation can be applied partion by partition, so the partitions can be processed locally on the same cluster node. Simple row-based filter rules or commutative operations like summing up values, are common examples of narrow transformations.

On the other hand, in a *wide* transformation input partitions contribute to multiple output partitions, so data needs to be shuffled across cluster nodes. Sorting and average calculation are common wide transformations across multiple partitions. During shuffling Spark writes results to disk, so wide transformations are not performed in-memory.

Ok, now we want to see some action and Spark to showusthe first 10 lines in our data file.

In [36]:
```
dfFlight.show(10)
```

```
+-----------------+-------------------+-----+
|DEST_COUNTRY_NAME|ORIGIN_COUNTRY_NAME|count|
+-----------------+-------------------+-----+
|    United States|            Romania|   15|
|    United States|            Croatia|    1|
|    United States|            Ireland|  344|
```

```
|            Egypt|      United States|   15|
|    United States|              India|   62|
|    United States|          Singapore|    1|
|    United States|            Grenada|   62|
|       Costa Rica|      United States|  588|
|          Senegal|      United States|   40|
|          Moldova|      United States|    1|
+-----------------+-------------------+-----+
only showing top 10 rows
```

Now we triggered actual data processing so we can see the results. Next to showing data, there are two other types of actions: writing output data, e.g. to file and actions to collect data to native objects in the respercitve language.

we can even combine transformations and actions in one single command.

In [37]:
```
spark.read\
   .option("inferSchema", "true")\
   .option("header", "true")\
   .csv("./data/flight-data/2015-summary.csv")\
   .show(10)
```

```
+-----------------+-------------------+-----+
|DEST_COUNTRY_NAME|ORIGIN_COUNTRY_NAME|count|
+-----------------+-------------------+-----+
|    United States|            Romania|   15|
|    United States|            Croatia|    1|
|    United States|            Ireland|  344|
|            Egypt|      United States|   15|
|    United States|              India|   62|
|    United States|          Singapore|    1|
|    United States|            Grenada|   62|
|       Costa Rica|      United States|  588|
|          Senegal|      United States|   40|
|          Moldova|      United States|    1|
+-----------------+-------------------+-----+
only showing top 10 rows
```

Obviously we can split up any sequence of transformations by asigning the intermedate `Dataframe` object to a variable and have a look into the intermediate results by calling the `show()` function on that `Dataframe` . This is a very nice feature foruswhen we need to debug complex analytical queries or ETL jobs which compile severeal subqueries together. If our subqueries provide the expected result, the bug must reside in the remaining part of our transormation logic so we can focus our analysis on that area.

Query Explain Plans

So we've learned so far, that we just need to define the business logic by concatinating transformation functions and Spark does the optimisation for us . Fortunately Spark givesusinsight, how it will perform our query by calling the `explain()` function.

we want to see, how Spark would **physically** execute the sorting, which is a wide transformation. Each step in the explain plan actually generate a new `DataFrame` .

In [38]:
```
dfFlight.sort("count").show()
```

```
+--------------------+-------------------+-----+
|   DEST_COUNTRY_NAME|ORIGIN_COUNTRY_NAME|count|
+--------------------+-------------------+-----+
|       United States|            Estonia|    1|
|              Kosovo|      United States|    1|
|              Zambia|      United States|    1|
|       United States|   Papua New Guinea|    1|
|               Malta|      United States|    1|
|       United States|          Gibraltar|    1|
|            Suriname|      United States|    1|
|       United States|            Croatia|    1|
|            Djibouti|      United States|    1|
|        Burkina Faso|      United States|    1|
|Saint Vincent and...|      United States|    1|
|       United States|             Cyprus|    1|
|       United States|          Singapore|    1|
|             Moldova|      United States|    1|
|              Cyprus|      United States|    1|
|       United States|          Lithuania|    1|
|       United States|           Bulgaria|    1|
|       United States|            Georgia|    1|
|       United States|            Bahrain|    1|
|       Cote d'Ivoire|      United States|    1|
+--------------------+-------------------+-----+
only showing top 20 rows
```

Reading the explain plan from bottom upwards, it tells us , that first, Spark performs a file scan and than range partitioning is applied shuffling the data over 200 output partitions by default to sort the data.

In [39]:
```
spark.conf.get("spark.sql.shuffle.partitions")
```

Out[39]:
```
'5'
```

Since we are running in local mode on a single machine, it might be better to limit the number of partitions to 5. we can do this be chaging the configuration of the `SparkSession` object *spark*.

In [40]:
```
spark.conf.set("spark.sql.shuffle.partitions", "5")
```

The explain plan confirms, that our configuration change has the desired effect.

In [41]:
```
dfFlight.sort("count").explain()
```

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Sort [count#381 ASC NULLS FIRST], true, 0
   +- Exchange rangepartitioning(count#381 ASC NULLS FIRST, 5), ENSURE_REQUIREMENTS, [plan_id=505]
      +- FileScan csv [DEST_COUNTRY_NAME#379,ORIGIN_COUNTRY_NAME#380,count#381] Batched: false, Da
taFilters: [], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/C:/Users/bda/OneDrive/BDA2/
Pyspark/pyspark_class/data/flight-dat..., PartitionFilters: [], PushedFilters: [], ReadSchema: str
uct<DEST_COUNTRY_NAME:string,ORIGIN_COUNTRY_NAME:string,count:int>
```

The `explain()` function can helpusfiguring out how flexible we can chain up functions which define `DataFrame` to `DataFrame` transformations. For example, is it relevant for the query execution, whether we filter before selecting or the other way around?

In [42]:

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()
dfRetail = spark.read\
   .format("csv")\
   .option("header", "true")\
   .option("inferSchema", "true")\
   .load('./data/retail-data/by-day/*.csv')
```

In [43]:

```
# import os
# # Define the directory containing the CSV files
# csv_directory = './data/retail-data/by-day/'

# # List all the CSV files in the directory
# csv_files = [f for f in os.listdir(csv_directory) if f.endswith('.csv')]

# # Read the first CSV file to create the initial DataFrame
# dfRetail = spark.read \
#     .option("header", "true") \
#     .option("inferSchema", "true") \
#     .csv(os.path.join(csv_directory, csv_files[0]))

# # Read the remaining CSV files and merge their content with the initial DataFrame
# for file in csv_files[1:]:
#     df_temp = spark.read \
#         .option("header", "true") \
#         .option("inferSchema", "true") \
#         .csv(os.path.join(csv_directory, file))
#     dfRetail = dfRetail.union(df_temp)

# Show the first 5 rows
dfRetail.show(5)
```

```
+---------+---------+--------------------+--------+-------------------+---------+----------+------
--------+
|InvoiceNo|StockCode|         Description|Quantity|        InvoiceDate|UnitPrice|CustomerID|
Country|
+---------+---------+--------------------+--------+-------------------+---------+----------+------
--------+
|   580538|    23084|  RABBIT NIGHT LIGHT|      48|2011-12-05 08:38:00|     1.79|   14075.0|United
Kingdom|
|   580538|    23077| DOUGHNUT LIP GLOSS |      20|2011-12-05 08:38:00|     1.25|   14075.0|United
Kingdom|
|   580538|    22906|12 MESSAGE CARDS ...|      24|2011-12-05 08:38:00|     1.65|   14075.0|United
Kingdom|
|   580538|    21914|BLUE HARMONICA IN...|      24|2011-12-05 08:38:00|     1.25|   14075.0|United
Kingdom|
|   580538|    22467|   GUMBALL COAT RACK|       6|2011-12-05 08:38:00|     2.55|   14075.0|United
Kingdom|
+---------+---------+--------------------+--------+-------------------+---------+----------+------
--------+
only showing top 5 rows
```

Can we get a performance benefit, when we filter very early?

In [44]:

```
from pyspark.sql.functions import col

dfRetail.where(col("InvoiceNo") != 536365)\
```

```
    .select("InvoiceNo", "Description")\
    .explain()
```

```
== Physical Plan ==
*(1) Filter (isnotnull(InvoiceNo#475) AND NOT (cast(InvoiceNo#475 as int) = 536365))
+- FileScan csv [InvoiceNo#475,Description#477] Batched: false, DataFilters: [isnotnull(InvoiceNo#
475), NOT (cast(InvoiceNo#475 as int) = 536365)], Format: CSV, Location: InMemoryFileIndex(249 pat
hs)[file:/C:/Users/bda/OneDrive/BDA2/Pyspark/pyspark_class/data/retail-d..., PartitionFilters: [],
PushedFilters: [IsNotNull(InvoiceNo)], ReadSchema: struct<InvoiceNo:string,Description:string>
```

Or can we stick to the well-known SQL pattern: SELECT ... FROM ... WHERE?

In [45]:
```
dfRetail.select("InvoiceNo", "Description")\
    .where(col("InvoiceNo") != 536365)\
    .explain()
```

```
== Physical Plan ==
*(1) Filter (isnotnull(InvoiceNo#475) AND NOT (cast(InvoiceNo#475 as int) = 536365))
+- FileScan csv [InvoiceNo#475,Description#477] Batched: false, DataFilters: [isnotnull(InvoiceNo#
475), NOT (cast(InvoiceNo#475 as int) = 536365)], Format: CSV, Location: InMemoryFileIndex(249 pat
hs)[file:/C:/Users/bda/OneDrive/BDA2/Pyspark/pyspark_class/data/retail-d..., PartitionFilters: [],
PushedFilters: [IsNotNull(InvoiceNo)], ReadSchema: struct<InvoiceNo:string,Description:string>
```

The execution plan is the same. Again Spark does the optimization in the background and performs the filter before the column projection. In this case, the functional PAI providesusmore flexibility than the strict SQL syntax.

Spark SQL, Tables and Views

SQL Language Reference provided by Databricks.

abc

we've been working with relational databases and SQL for many years. So we are happy to notice, that Spark also speaks our languange. In fact, the Spark SQL API supports the ANSI SQL 2003 standard. we can turn a `Dataframe` into a table or view, which we can query with SQL. All we need to do is register a table/view on that `Dataframe` .

In [46]:
```
dfFlight.createOrReplaceTempView("flight_data_2015")
```

Now we can write our Spark query as we did it for long times in classical databases, and we will get exactly the same result, as doing it the functional way.

This example calculates the top 5 countries having the highest number of flight destinations in 2015. Obviously the most flights went to the US.

In [47]:
```
# finding the top five destination countries by SQL
from pyspark.sql.functions import max, desc
# transformation
maxSql = spark.sql("""SELECT DEST_COUNTRY_NAME, sum(count) as destination_total
FROM flight_data_2015
GROUP BY DEST_COUNTRY_NAME
```

```
ORDER BY sum(count) DESC
LIMIT 5""")
# action
maxSql.show()
```

```
+-----------------+-----------------+
|DEST_COUNTRY_NAME|destination_total|
+-----------------+-----------------+
|    United States|           411352|
|           Canada|             8399|
|           Mexico|             7140|
|   United Kingdom|             2025|
|            Japan|             1548|
+-----------------+-----------------+
```

The equivalent functional query looks like this:

In [48]:

```
#transformation
dfFlight\
   .groupBy("DEST_COUNTRY_NAME")\
   .sum("count")\
   .withColumnRenamed("sum(count)","destination_total")\
   .sort(desc("destination_total"))\
   .limit(5)\
   .show()
```

```
+-----------------+-----------------+
|DEST_COUNTRY_NAME|destination_total|
+-----------------+-----------------+
|    United States|           411352|
|           Canada|             8399|
|           Mexico|             7140|
|   United Kingdom|             2025|
|            Japan|             1548|
+-----------------+-----------------+
```

The functional version looks touseven more self-explaing the **logical** transformations. The story reading it line-by-line is: first the data is grouped by the destination countries. Than, for each group (partition) the number of flights are summed up which generates a new, derived column which is than renamed. Afterwards the results are sorted is descending order by the calculated column and the output is limited by the top 5 rows.

Fortunately the convenience of using Spark SQL API instead of functions does not have an negative performance impact. The **physical** execution explain plans of both versions are exactly the same.

In [49]:

```
# both transformartions compile to the same plan
maxSql.explain()
```

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- TakeOrderedAndProject(limit=5, orderBy=[destination_total#537L DESC NULLS LAST], output=[DEST_C
OUNTRY_NAME#379,destination_total#537L])
   +- HashAggregate(keys=[DEST_COUNTRY_NAME#379], functions=[sum(count#381)])
      +- Exchange hashpartitioning(DEST_COUNTRY_NAME#379, 5), ENSURE_REQUIREMENTS, [plan_id=665]
         +- HashAggregate(keys=[DEST_COUNTRY_NAME#379], functions=[partial_sum(count#381)])
            +- FileScan csv [DEST_COUNTRY_NAME#379,count#381] Batched: false, DataFilters: [], For
mat: CSV, Location: InMemoryFileIndex(1 paths)[file:/C:/Users/bda/OneDrive/BDA2/Pyspark/pyspark_cl
ass/data/flight-dat..., PartitionFilters: [], PushedFilters: [], ReadSchema: struct<DEST_COUNTRY_N
AME:string,count:int>
```

In [50]:
```
dfFlight\
   .groupBy("DEST_COUNTRY_NAME")\
   .sum("count")\
   .withColumnRenamed("sum(count)","destination_total")\
   .sort(desc("destination_total"))\
   .limit(5)\
   .explain()
```

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- TakeOrderedAndProject(limit=5, orderBy=[destination_total#589L DESC NULLS LAST], output=[DEST_C
OUNTRY_NAME#379,destination_total#589L])
   +- HashAggregate(keys=[DEST_COUNTRY_NAME#379], functions=[sum(count#381)])
      +- Exchange hashpartitioning(DEST_COUNTRY_NAME#379, 5), ENSURE_REQUIREMENTS, [plan_id=682]
         +- HashAggregate(keys=[DEST_COUNTRY_NAME#379], functions=[partial_sum(count#381)])
            +- FileScan csv [DEST_COUNTRY_NAME#379,count#381] Batched: false, DataFilters: [], For
mat: CSV, Location: InMemoryFileIndex(1 paths)[file:/C:/Users/bda/OneDrive/BDA2/Pyspark/pyspark_cl
ass/data/flight-dat..., PartitionFilters: [], PushedFilters: [], ReadSchema: struct<DEST_COUNTRY_N
AME:string,count:int>
```

Interesting to note is that the `sum()` aggregation involves two *hashAggregate* steps. Because `sum()` is a commutative operation Spark first calculates partial sums partition by partition which is a narrow transformation. Afterwards the aggregted, i.e. already reduced data is shuffled (*Exchange hashpartitioning*) to calculate the overall sum across all partitions. This is another example how Spark optimizes the query execution by first analyzing all transformations befor starting data processing.

Spark Datasets

Datasets are a type-safe version of DataFrames. Since Python is a dynamically taped language, they are not available in pyspark but they can be used in the Java and Scala API. Good to keep in mind, but we skip it for now, since we prefer Python.

Structured Streaming

So far we did all data processing in batch mode, i.e. all data get's processed at once. Batch mode forcesusto wait, until all data we want to analyse is available. Stream processing on the other hand enablesusto process data incrementally as it arrives, so we can get insights faster.

Stream processing in Spark is very similar to data processing in batch mode. The following example will demonstrate this. As far as we can see so for now, this is because Spark stream processing is actually event-triggered mirco-batch-processing.

Ok, let's have a closer look and start from scratch, i.e. creating a new `SparkSession`.

In [51]:
```
# import pyspark
# from pyspark.sql import SparkSession
from pyspark.sql.functions import window, column, col, sum

# spark = SparkSession.builder.getOrCreate()
```

```
# spark = SparkSession.builder.master("local").config("spark.driver.memory", "8g").getOrCreate()
spark.conf.set("spark.sql.shuffle.partitions", "5")
```

Batch processing

This time, our data source is not a single file, instead the data is split into several files on a day-by-day basis. Nevertheless, infering the schema, the us ta data, works the same way. The only difference is the wildcard * in the filename to tell Spark, that we want to process all CSV files in the specified folder.

In [52]:
```
# list1=["./data/retail-data/by-day/2010-12-01.csv","./data/retail-data/by-day/2010-12-02.csv"]
staticDF = spark.read\
    .format("csv")\
    .option("header", "true")\
    .option("inferSchema", "true")\
    .load("./data/retail-data/by-day/*.csv")
```

This defines the transformation which tells Spark how to create the source DataFrame.

As a retailer, we want to analyse how much money each customer is pending in our shops per hour in each 1 day time window. So we add a further transformation on that `DataFrame`, which describes the business logic of our data analysis, and results to another `DataFrame`.

In [53]:
```
purchaseByCustomerPerHour = staticDF\
    .selectExpr("CustomerId", "(UnitPrice * Quantity) as total_cost", "InvoiceDate")\
    .groupBy("CustomerId", window("InvoiceDate", "1 day"))\
    .sum("total_cost")
```

To take a look at the first 10 rows of the result, we have to call the action `show()`.

In [54]:
```
purchaseByCustomerPerHour.show(10)
```

```
+----------+--------------------+------------------+
|CustomerId|              window|   sum(total_cost)|
+----------+--------------------+------------------+
|   14075.0|{2011-12-04 19:00...|316.78000000000003|
|   18180.0|{2011-12-04 19:00...|            310.73|
|   15358.0|{2011-12-04 19:00...| 830.0600000000003|
|   15392.0|{2011-12-04 19:00...|304.40999999999997|
|   15290.0|{2011-12-04 19:00...|263.02000000000004|
|   16811.0|{2011-12-04 19:00...|             232.3|
|   12748.0|{2011-12-04 19:00...| 363.7899999999999|
|   16500.0|{2011-12-04 19:00...| 52.74000000000001|
|   16873.0|{2011-12-04 19:00...|1854.8300000000002|
|   14060.0|{2011-12-04 19:00...|297.47999999999996|
+----------+--------------------+------------------+
only showing top 10 rows
```

In batch mode, we are actually processing the entire data history, i.e. all files at once, which can take quite a long time for large data sets. To make this faster, we can switch to stream processing.

Stream processing

There are just two things, we have to do, to turn our batch processing into stream processing in Spark:

- using `readStream()` instead of `read()`, and
- defining a trigger refreshing the result after reading each input file

In the given example, a trigger get's fired after reading each file (*maxFilesPerTrigger* = 1). Since all files are already on our harddrive, Spark will actually refresh the results every few (milli-)seconds, so finally we are quite close to realtime-processing in this demonstration.

The schema is the same, as for batch processing, so we are re-using it from the *staticDF*.

In [55]:
```
# import findspark
# findspark.init()

# from pyspark.sql import SparkSession

# spark = SparkSession.builder \
#     .appName("YourAppName") \
#     .getOrCreate()
```

In [56]:
```
streamingDF = spark.readStream\
    .schema(staticDF.schema)\
    .option("maxFilesPerTrigger", 1)\
    .format("csv")\
    .option("header", "true")\
    .load("./data/retail-data/by-day/*.csv")
```

Let's check, if the Stream creation was successfully.

In [57]:
```
streamingDF.isStreaming
```

Out[57]:
```
True
```

The transformation is still the same, but now it is applied on a stream instead of a DataFrame.

In [58]:
```
purchaseByCustomerPerHour = streamingDF\
    .selectExpr("CustomerId", "(UnitPrice * Quantity) as total_cost", "InvoiceDate")\
    .groupBy(col("CustomerId"), window(col("InvoiceDate"), "1 day"))\
    .sum("total_cost")
```

As we've learnd so far, That Spark evaluates lazly and nothing happens untill we call an action to initiate the stream processing. The action `writeStream` generates a table, which gets updated after each trigger event. Important to note is, **streaming tables are mutable** whereas `DataFrame` objectss **are immutable.**

Here we stream the results to our console using `format("console")`, to make it visible how the result table gets updated regularly. Using `format("memory")` would push the stream to an in-memory table so other stream processes could read it.

In [59]:
```
purchaseByCustomerPerHour\
    .writeStream\
    .format("console")\
    .queryName("customer_purchases")\
    .outputMode("complete")\
    .start()
```

Out[59]: <pyspark.sql.streaming.query.StreamingQuery at 0x14e8bbd3520>

In [ ]: